

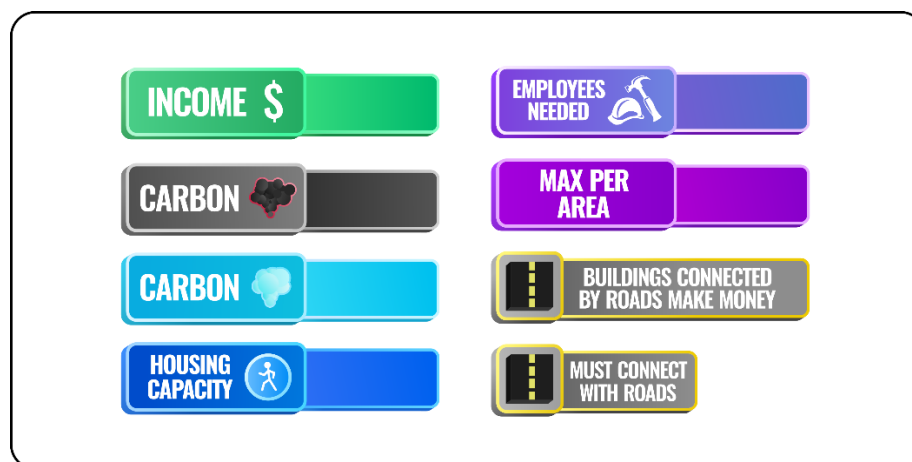
TOOL TIP SYSTEM

OVERVIEW

The tool tip system manages the informational tool tips that appear over tile buttons (on the far right of the screen). The primary scripts behind the system are `ToolTipManager.cs` and `ToolTipFormatting.cs`. The abstract class `ToolTipType` is also heavily used because each bit of information that can appear in a tool tip gets its own class that inherits from `ToolTipType`.

HOW THE TOOL TIP SYSTEM WORKS

The entire tool tip system sits under the `ToolTipManager` game object, which has the `ToolTipManager` script attached to it. The first objects under the `ToolTipManager` game object are some visual elements that compose the background of the tool tips; after them, there's a list of game objects that compose all the different pieces of information that can be displayed by a tool tip. Each piece of information that can appear on a tool tip gets its own script and game object. I'll refer to these as tool tip elements. Here's what some of them look like:





Whenever the player begins hovering over a tile button, the button passes itself and the tile it places to the tool tip manager and tells the manager to activate the tool tip. The tool tip manager will then check if each tool tip element should be activated for the given button, by calling the `ShouldEnableToolTip(Tile tile)` function for each tool tip element, passing the Tile script of the tile that the button places. If the tool tip element should be enabled, it calls the `EnableToolTip(Tile tile)` function for the tool tip. The `ShouldEnableToolTip` and `EnableToolTip` functions use the tile's scriptable object to determine if the tool tip element should be activated and what information should be displayed.

Once all of the tool tip elements that should be activated are activated, the tool tip manager then uses the `ToolTipFormatter` class to arrange all of the elements and correctly position the tool tip over the right button.

HOW TO ADD A NEW TOOL TIP ELEMENT

1. Create a new tool tip element game object (I would recommend duplicating one of the other ones, like “MoneyToolTip”) and make it a child of the ToolTipManager game object.
 - a. Swap out the graphics with your own.
2. Create a class that inherits from ToolTipType and put it in a new script. Store the script in this location:
Assets/Scripts/UIScripts/ToolTips/TypesOfToolTips
 - a. Attach the script to the tool tip element game object you created in step one.
3. In the script you created, add override functions for ShouldEnableToolTip() and EnableToolTip(). They dictate when the tool tip should be turned on and what text they should display when turned on, respectively.
 - a. To write the functions, I would recommend copying the functions from the MoneyToolTip.cs class and modifying them as needed.
4. In the Inspector, add a new entry to the ToolTipManager’s “allToolTips” array. Drag in your new tool tip into the new slot.

Note: The tool tips will be displayed in the order they appear in the “allToolTips” array in the inspector under the ToolTipManager class.

INTERNAL API

ToolTipManager.cs

The Tool Tip Manager, which is a singleton, can be accessed like this: ToolTipManager.TTM

```
public void activateToolTip(Tile tile, GameObject button)
```

Activates the tool tip for the input tile's scriptable object. It then positions the tool tip next to the input "button" game object.

```
public void deactivateToolTip()
```

Turns off the tool tip.

```
public void updateToolTipPosition()
```

Moves the tool tip to be next to the active button. Called whenever the tile select panel scrolls.

ToolTipFormatting.cs

```
public void FormatToolTip(RectTransform[] toolTipItems)
```

Formats the tool tip by correctly spacing all of elements in "toolTipItems". It also correctly sizes the background of the tool tip based on the total height of all the items in "toolTipItems".

-Graydon Bruyn, May 2026